Build a secure, accessible, production-ready web application called “ClearCourse AI.” It is a learning-transparency platform for higher education that combines Course AI Cards, Assignment AI Cards, an anonymous and upvotable question board, optional process-transparency bonuses, classroom presentation mode, and privacy-preserving participation verification.

Implement the complete application, database, authentication, authorization, moderation, audit logging, automated tests, deployment documentation, and administrative configuration. Do not build AI detection, automated misconduct prediction, sentiment analysis, automated quality scoring, or student-risk profiles.

INSTITUTIONAL CONTEXT

The initial deployment is intended for the University of Waterloo and should be capable of linking to LEARN, Policy 71, and the Guidelines for the Assessment of Penalties. All institutional names, links, policy language, branding, and LMS configuration must be administrator-configurable so the application can later support other institutions.

Default University of Waterloo policy links:

Policy 71:

https://uwaterloo.ca/secretariat/policies-procedures-guidelines/policy-71

Guidelines for the Assessment of Penalties:

https://uwaterloo.ca/secretariat/policies-procedures-guidelines/guidelines/guidelines-assessment-penalties

The application must not present legal or policy summaries as authoritative replacements for the linked official policies.

ROLES AND AUTHORIZATION

Support:

- Student
- Instructor
- Teaching assistant
- Institutional administrator

Use strict role-based and course-based authorization.

Students may access only courses in which they are enrolled.

Instructors may manage only courses they teach.

Teaching assistants receive specific permissions assigned by instructors, including optional moderation and card-editing privileges.

Institutional administrators may manage institutional configuration but must not be able to routinely connect anonymous content with student identities.

Do not implement impersonation without a tightly controlled, audited support procedure.

PRIVACY ARCHITECTURE

Design anonymity as an architectural property, not merely a hidden name in the interface.

Separate:

1. Anonymous discussion content
2. Private per-student participation counters
3. Student identity and enrollment
4. Instructor-visible shared participation results
5. Anonymous exit-survey responses

Instructors and teaching assistants must never receive access to mappings between students and anonymous questions, comments, or upvotes.

The application may update a student's private participation counter when a qualifying action occurs, but ordinary application queries must not allow the action content to be joined back to the student identity.

Use privacy-preserving event processing, separated tables or schemas, restricted service roles, opaque identifiers, and narrowly scoped internal APIs. Document the threat model and the limitations of anonymity.

Do not place identity-bearing information and anonymous-content text in the same application log, analytics event, error report, database view, export, or administrator dashboard.

For abuse investigations, prefer content moderation and rate controls that do not reveal identity. If institutional policy requires an exceptional deanonymization capability, do not enable it silently. Implement it only as an institution-configurable exceptional process requiring:

- Explicit institutional approval
- Multiple authorized approvers
- A documented reason
- Immutable audit logging
- Notification rules
- A published explanation visible to users

The default configuration should not permit instructors, TAs, or ordinary administrators to deanonymize content.

CORE COURSE STRUCTURE

Each course has:

1. One Course AI Card
2. One AI Card per assignment
3. One anonymous question board

QUESTION CATEGORIZATION

Questions have exactly two tag types:

1. Course

2. Specific Assignment

There is no third “general” tag.

The unfiltered board shows all course and assignment questions.

Filters include:

- All questions
- Course questions
- One selectable assignment

COURSE AI CARD

Support these fields:

- What counts as generative AI
- Whether generative AI is generally permitted
- Uses generally allowed
- Uses generally prohibited
- General disclosure requirements
- General transparency-of-process expectations
- How suspected inappropriate use is addressed
- Configurable institutional-policy warning
- Policy 71 link
- Guidelines for the Assessment of Penalties link
- Additional resources
- Last updated date
- Version history
- Published status

If AI is generally prohibited, allow language explaining that drafts or version history may be requested if concerns arise.

If AI is permitted, require the instructor to specify permitted uses, prohibited uses, and disclosure expectations.

Include the default principle:

“Permission to use generative AI is not a recommendation or requirement to use it. Students will not receive an advantage merely for using AI. Any transparency bonus rewards evidence and reflection about the learning process, whether that process involved AI or not.”

ASSIGNMENT AI CARDS

Support:

- Assignment title
- Assignment identifier
- Due date
- Learning being assessed
- Is AI permitted?
- When AI is allowed
- When AI is not allowed
- When disclosure is required
- When disclosure is not required
- When process evidence is requested
- When process evidence is not requested
- Optional transparency bonus percentage
- Requirements for receiving the transparency bonus

- Resources
- Other optional information
- LEARN submission link
- Policy links
- Draft, published, archived, and superseded states
- Version history
- Published date
- Last updated date

Each card should begin with a configurable statement modeled on:

“This assignment does not require you to use AI. If you choose to use it, the rules below explain what is permitted and what must be disclosed. The optional transparency-of-process bonus is available equally to students who use AI and students who do not.”

Assignment rules override the Course AI Card when they differ. Display differences visibly.

Do not accept assignment submissions in this application. Direct students to the configured LEARN assignment-submission URL.

TRANSPARENCY-OF-PROCESS BONUS

The bonus is always optional and available equally to students who use AI and students who do not.

Allow instructors to configure the percentage and the evidence requested.

If AI was used, recommended evidence includes:

- Selected prompts and responses copied into a document; or
- A PDF of the relevant AI conversation
- Permission to blur or black out personal, unrelated, or private text
- Draft or version history, when applicable
- A process reflection of approximately 100 words explaining the student's actual process, use of AI, evaluation of responses, decisions, verification, and response to feeling stuck

If AI was not used, recommended evidence includes:

- Planning notes
- Evidence of feedback or revision
- Draft or version history
- A process reflection of approximately 100 words explaining the student's process, challenges, writer's block or difficulty beginning, resources used, and important revisions

If AI is not permitted, the recommended optional transparency evidence is:

- Drafts or version history
- A brief process reflection explaining how the student completed the work and navigated difficulty

All files and reflections are submitted through LEARN. Store only the requirements and links in ClearCourse AI.

INSTRUCTOR CARD-BUILDER

Implement a guided wizard and a manual editor.

The wizard asks:

- What learning is being assessed?
- Which work must students perform independently?
- Could AI assist without replacing that learning?
- Is AI permitted?
- At which stages?
- What is prohibited?
- What must be disclosed?
- What process evidence could support reflection?
- Is a transparency bonus offered?
- What is its value?
- What resources should students receive?
- What happens if inappropriate use is suspected?

For each question, provide:

- A short explanation
- “Why this matters”
- Curated supporting resources
- The transparent decision rule behind any recommendation

Generate a recommended card using explicit fixed rules. Do not use a generative model to set policy.

Instructors may:

- Edit every field
- Reject recommendations

- Add custom language
- Skip the wizard
- Preview the student view
- Save drafts
- Publish
- Duplicate cards
- Archive or supersede cards

Version every published card. Preserve the rules that were visible to students at each point in time.

If a published rule changes, require the instructor to specify:

- What changed
- Why it changed
- Effective date
- Whether the change applies to work already underway

Notify affected students.

ANONYMOUS QUESTION BOARD

Students can:

- Ask an anonymous question
- Comment anonymously
- Upvote a question
- Mark answer “Still unclear”
- Report spam or inappropriate content
- Follow a question

Instructors and authorized TAs can:

- Answer
- Pin
- Merge duplicates
- Recategorize between Course and a specific assignment
- Hide or delete content
- Resolve or reopen
- Select for presentation mode

When duplicate questions are merged:

- Preserve all comments where reasonable
- Combine upvote totals without double-counting a student
- Preserve qualifying private participation credit
- Redirect old links to the merged question
- Log the moderation action without exposing identities

Implement rate limiting, duplicate warnings, safe content reporting, and a moderation queue. Do not use automated semantic judgment to grade or evaluate student participation. If duplicate suggestions are implemented, treat them only as optional search assistance and never deny participation automatically.

CLASSROOM PRESENTATION MODE

Display:

- Anonymous question
- Upvote total
- Answer
- Relevant card section
- Clarification or policy-change status
- Discussed-in-class status

Exclude:

- Identities
- Private participation counts
- Student activity breakdowns
- Moderator-only notes
- Reports
- Hidden comments

PRIVATE PARTICIPATION COUNTER

Count only:

- Questions asked
- Comments posted
- Upvotes given

Do not count views, reports, follows, card opens, time spent, or survey responses.

Provide exactly three participation levels:

1. No Record
2. Contributor
3. Sustained Contributor

Recommended default thresholds:

Contributor:

- At least one question or comment
- At least one upvote

Sustained Contributor:

- At least two questions or comments
- At least three upvotes

No Record:

- Contributor threshold not met

Allow instructors to configure the numerical thresholds before publishing the participation policy. Once students begin participating, prevent threshold changes that disadvantage them. Preserve the originally published threshold version.

Do not use AI, quality scoring, sentiment analysis, instructor ratings, or number of upvotes received to determine a student's level.

STUDENT-CONTROLLED SHARING

Students privately see:

- Question count
- Comment count
- Upvote count
- Current level
- Published course thresholds

At the end of the course, students may share only their verified level.

Before sharing, show:

“Your instructor will receive your name and participation level. Your instructor will not receive your questions, comments, upvotes, dates, or activity breakdown.”

The instructor receives only:

- Student name
- Verified level
- Date shared

Support revocation before the instructor exports or applies the result. Clearly explain when sharing becomes final.

INSTRUCTOR GRADING SETTINGS

Support:

1. Pass–fail
2. Ranked

Pass–fail:

- Instructor chooses Contributor or Sustained Contributor as the passing threshold.

Ranked:

- Instructor assigns a published bonus or participation value to each level.

Provide recommended defaults. Require instructors to publish the scheme before students participate. Do not automatically determine the final course grade; provide an authorized grade result or export compatible with institutional requirements.

END-OF-COURSE SURVEY

Before participation-level sharing, present three anonymous questions:

1. What should the next version of this course do differently to support transparency and your learning process?
2. What, if anything, remained confusing even after questions were answered?
3. When did you feel best supported through this platform, and what was helpful?

Requirements:

- Anonymous
- Technically separated from identity and participation records
- Available even if the student does not share their level
- Instructor sees only aggregate counts and anonymous text
- Apply a configurable minimum-response threshold before revealing written responses in small classes
- Do not collect unnecessary metadata
- Do not award participation-counter credit for completing it

NOTIFICATIONS

Support accessible in-app and email notifications for:

- Answer posted
- Followed question updated
- Assignment card changed
- Course card changed
- Question merged
- Participation level reached
- End-of-course survey available
- Sharing window open

Allow users to manage nonessential notifications.

SEARCH AND FILTERING

Support:

- Search within the current course
- All questions
- Course questions
- Specific assignment questions
- Unanswered
- Answered
- Pinned
- Resolved
- Most upvoted
- Most recent

Search results must not expose hidden, removed, moderator-only, or identity-linked information.

ADMINISTRATION

Allow institutional administrators to update website

ANALYTICS

Provide only privacy-preserving course aggregates, such as:

- Total questions
- Total comments
- Total upvotes
- Unanswered-question count
- Questions by Course versus assignment
- Percentage of questions resolved
- Number of students in each private level, shown only when aggregation thresholds are met
- Exit-survey aggregate results

Do not provide student behavioral timelines, risk scores, identity-linked discussion analytics, or instructor access to individual activity breakdowns.

TESTING

Include explicit negative tests proving that instructors and TAs cannot retrieve the authors of anonymous content through the UI, API, exports, logs, or predictable identifiers.

DESIGN

Use a very minimal simple, calm, credible academic design. Avoid AI-generated aesthetics, robots, surveillance motifs, leaderboards, streaks, competitive badges, and attention-maximizing patterns. Take learn/brightspace/uw flow as inspiration

Use:

- Compact information cards
- Plain language
- Strong hierarchy

- University-appropriate typography
- Neutral colours with configurable institutional branding
- Progressive disclosure
- Clear status labels
- Explicit anonymity explanations
- Non-gamified private participation levels

PRODUCTION ACCEPTANCE CRITERIA

The completed application must demonstrate that:

- Institutional users can sign in securely
- Courses and roles can be provisioned through supported integrations
- Instructors can create, edit, preview, publish, and version course and assignment cards
- Students can clearly distinguish course rules from assignment overrides
- Questions use only Course or Specific Assignment categorization
- Students can ask, comment, and upvote anonymously
- Instructors and TAs can moderate without accessing identities
- Duplicate merging preserves counts correctly
- Presentation mode reveals no private data
- Participation uses only question, comment, and upvote counts
- Exactly three levels are supported
- Students alone decide whether to share their level
- Instructors receive no underlying participation activity
- The exit survey is anonymous and technically separate
- No AI detection or automated student judgment exists

summarize the doc above ^

from a technical perspective what is the whole scope and product view

From a technical perspective,

ClearCourse AI is designed as a secure, role-based web application with a strict focus on privacy and anonymity.

Here is a breakdown of the system architecture and product scope:

Prompt

1. Application and Infrastructure Scope

The platform requires a complete technical implementation, which includes:

Prompt

Core Systems: Full web application, database, authentication, authorization, moderation, and audit logging.

Prompt

DevOps: Automated tests and deployment documentation.

Prompt

Administrative Controls: Institutional configuration settings, allowing administrators to customize branding, policy links, and Learning Management System (LMS) integrations.

Prompt

Explicit Exclusions: The system must not include any generative AI components that make automated judgments about students. This means no AI detection, automated misconduct prediction, sentiment analysis, quality scoring, or risk profiling.

Prompt

2. Authorization and Access Control

The application uses strict role-based and course-based authorization with four primary roles:

Prompt

Students: Can only access courses they are actively enrolled in.

Prompt

Instructors: Can only manage the specific courses they teach.

Prompt

Teaching Assistants: Receive granular permissions assigned by the instructor, such as moderation or card-editing privileges.

Prompt

Institutional Administrators: Manage global configurations but are explicitly blocked from routinely connecting anonymous discussion content to student identities.

Prompt

Impersonation: Account impersonation is prohibited without a tightly controlled and audited support procedure.

Prompt

3. Privacy and Anonymity Architecture

Anonymity is a foundational architectural property, not just a UI toggle. The system must technically separate:

Prompt

Anonymous discussion content.

Prompt

Private per-student participation counters.

Prompt

Student identity and enrollment data.

Prompt

Instructor-visible shared participation results.

Prompt

Anonymous exit-survey responses.

Prompt

Technical Constraints:

Event Processing: The platform must use privacy-preserving event processing, separated database tables or schemas, restricted service roles, opaque identifiers, and narrowly scoped internal APIs to prevent data joining.

Prompt

Logging: Identity-bearing information and anonymous text must never coexist in the same application log, analytics event, database view, export, or administrator dashboard.

Prompt

Deanonymization: A highly exceptional deanonymization capability can only be implemented if institutional policy requires it. This requires explicit approval, multiple authorized approvers, immutable audit logging, and a published explanation visible to users. Instructors and TAs cannot deanonymize content by default.

Prompt

4. Core Product Features

The application is structured around three main components per course:

Prompt

Course and Assignment AI Cards: A policy management system where instructors define permitted AI uses, disclosure requirements, and transparency bonuses. This includes an "Instructor Card-Builder" wizard that generates rules based on explicit logic, not a generative model.

Prompt

Anonymous Question Board: A moderation-controlled discussion forum with upvoting, duplicate merging (which preserves participation counts without double-counting), and course/assignment tagging.

Prompt

Classroom Presentation Mode: A display view that shows questions and answers but strips all identities, private participation counts, and moderator notes.

Prompt

5. Participation Tracking and Analytics

Private Counters: The system counts only questions asked, comments posted, and upvotes given (ignoring views, time spent, or survey responses).

Prompt

Sharing: Students privately view their count and decide at the end of the course whether to share their final "tier" (No Record, Contributor, or Sustained Contributor) with the instructor. The instructor receives only the name and tier, not the underlying activity.

Prompt

Analytics: Dashboards provide only privacy-preserving aggregates (e.g., total questions, percentage resolved). Student behavioral timelines and identity-linked analytics are strictly prohibited.

Prompt

Was this visual helpful?

So it should be a web app that's hosted on a public domain, and then it needs to have walkthrough the database parts, the authentication parts that would be needed, and just how the web app is actually composed and the tech stack involved.

Here is the recommended technical scope and architecture to meet the strict privacy and institutional requirements of ClearCourse AI.

Recommended Tech Stack

Frontend: Next.js (React) with Tailwind CSS. This ensures a fast, accessible, and clean user interface that supports the required "credible academic design" without gamification.
Backend: Node.js with TypeScript (Express or NestJS) or Python (FastAPI). These frameworks support building narrowly scoped internal APIs necessary for strict data separation.
Database: PostgreSQL. A relational database is mandatory here to enforce strict schema separation, row-level security, and complex role-based authorization.
Hosting and Deployment: Cloud provider like AWS, Google Cloud, or Microsoft Azure. The frontend can be hosted on a CDN (like Vercel or AWS CloudFront) on a public domain, while the backend runs in isolated Docker containers (e.g., AWS ECS or Google Cloud Run).

Database Architecture

To satisfy the requirement that anonymity is an "architectural property," the database must be split into isolated schemas. A single query must not be able to join a user's identity to their anonymous posts.

Identity Schema: Stores user profiles, real names, emails, and course enrollment mapping.

Content Schema: Stores course AI cards, assignment rules, and the anonymous question board (posts, comments, upvotes). Posts are linked only to a generic course ID, never to an author ID.
Participation Schema: Stores opaque participation counters. When a user posts in the Content Schema, an event triggers an update to a counter in this schema using a secure, one-way hashed identifier.
Audit Schema: An immutable log tracking configuration changes, policy updates, and any highly restricted deanonymization events.

Authentication (AuthN)

Because this is an institutional application, you should not build a custom password system.

Single Sign-On (SSO): Integrate with the institution's existing identity provider using SAML or OAuth 2.0. For the University of Waterloo, this means integrating with Microsoft Entra ID (formerly Azure AD) or WatIAM.
Session Management: Secure, HTTP-only cookies storing JWTs (JSON Web Tokens) to maintain the user's session securely.

Authorization (AuthZ)

Authorization must be handled at the API route level using strict Role-Based Access Control (RBAC).

Middleware Checks: Every API request must pass through middleware that verifies the user's role (Student, TA, Instructor, Admin) and their explicit enrollment status in the requested course.
Granular Permissions: TAs require dynamic permission checks because instructors can assign them specific privileges, such as moderation or card editing, on a per-course basis.

Was this visual helpful?

How would the hosting be set up? Would that be able to be set up by Cloudflare, or would I have to set up AWS myself? How does that work for hosting? And once it's hosted, is everything else handled there? And how much would that be price-wise and work-wise?

You can set this up using either Cloudflare or AWS, but they require entirely different levels of effort and have different pricing models. Here is how hosting works for this type of application and what it takes.

- **How Hosting Works**

Once an app is hosted, the cloud provider manages

the physical servers, network routing, and uptime. You do not buy hardware. However, depending on the provider, you are still responsible for configuring the database, securing the network rules, deploying updates, and managing backups. It is not completely hands-off unless you use a fully managed "Platform as a Service" (PaaS).

- **Option 1: The Cloudflare Ecosystem (Low Effort, Low Cost)**

  You can build and host the entire application directly on Cloudflare.

  Setup: You use Cloudflare Pages to host the Next.js frontend, Cloudflare Workers for the backend API, and Cloudflare D1 for the SQL database. Everything is deployed automatically when you push code to GitHub.
  Operational Work: Very low. Cloudflare manages scaling automatically. There are no virtual servers or network configurations to maintain.

Price: Cloudflare has a generous free tier. To get production limits and D1 database access, you need the Workers Paid plan, which is a flat $5/month. This covers 10 million backend requests, 25 billion database reads, and 5 GB of database storage. Costs only increase slightly if you exceed those massive limits.

- **Option 2: AWS (High Effort, Variable Cost)**

AWS requires you to piece together different infrastructure components manually.

Setup: You host the frontend on AWS Amplify or S3/CloudFront, run the Node.js backend in Docker containers using AWS App Runner or ECS, and provision a managed PostgreSQL database using Amazon RDS.
Operational Work: High. You must configure Virtual Private Clouds (VPCs), set up security groups (firewall rules) so the backend can talk to the database, manage database connections, and monitor scaling policies.
Price: Much more expensive for small apps. A minimal production ready setup (a small App Runner instance and a basic `db.t3.micro` RDS database) will start around $30 to $50 per month. If you want high availability (Multi-AZ) for the database to ensure zero downtime, RDS costs jump to over $70/month immediately.

For a university tool like ClearCourse AI where budget might be tight and you want minimal maintenance, the

Cloudflare stack (Pages + Workers + D1) or a modern PaaS like Vercel paired with a Supabase database is highly recommended over bare AWS.

Was this visual helpful?

What are other options? I just said Cloudflare and AWS, but what are the actual best options just for even like a prototype of this but to be hosted publicly?

The best options for hosting a Next.js and PostgreSQL prototype without the operational overhead of AWS are modern Platform as a Service providers. These platforms connect directly to your GitHub repository and handle the build process, SSL certificates, and

public domains automatically.

Vercel combined with Supabase (Lowest Cost): You host the Next.js frontend on Vercel and use Supabase for the PostgreSQL database and authentication. Both have generous free tiers. Vercel is free for hobby projects. The Supabase free tier provides a 500 MB Postgres database and up to 50,000 monthly active users. The database will pause after 7 days of inactivity on the free tier, but it is perfect for a zero cost public prototype.

Railway (Easiest All In One Setup): Railway is a visual infrastructure platform where you can deploy your frontend, backend API, and PostgreSQL database in a single project canvas. It uses a pay as you go model starting at $5 per month, which includes a $5 usage credit. It analyzes your code and builds it automatically without requiring Docker configurations. An always on prototype will typically cost between $10 and $20 per month based on memory usage.

Render (Predictable Flat Pricing): Render acts as a much simpler alternative to AWS. You can deploy your web services and managed PostgreSQL databases from a unified dashboard. While they offer a free tier, the free database expires after 30 days and the web services spin down after 15 minutes of inactivity. For a permanent public prototype, you would use their lowest paid tiers, costing exactly $13 per month (7 for the web service and $6 for the database).

-